

Lloyds Bank

Economic Crime Prevention Unit Technology Hackathon Challenge

Challenge Title

"AlertDesk" — Economic Crime Case Management Platform

Role: Full Stack Engineer

Duration: 4 hours (1 × 60 min design sprint + 2 × 90 min dev sprints)

Primary Stack: Java 21 (Spring Boot 3) | REST APIs | Bean Validation | Docker

Stretch Stack: React JS | API integration

This document is strictly confidential. It is intended solely for hackathon participants. Do not share, copy, or distribute.

Scope Summary

Build the Java backend services first. Test locally. The React UI is a stretch goal attempted only after the backend is running.

Tier	What it covers	Sprint
Must Have	Alert Service + Case Management Service — validation, filtering, audit trail.	Sprints 1 + 2
Optional	Docker Compose running locally. Swagger docs.	Sprint 2 buffer
Stretch Scope	React front-end — Alert Queue and Case Detail views	Only after backend complete

1. Background & Problem Statement

Lloyds Bank's Economic Crime Prevention analysts manage fraud alerts and investigations across a patchwork of legacy tools. There is no unified system — cases lack audit trails, analysts cannot search data flexibly, and there is no API that enforces data quality at entry.

Your challenge is to build two Java Spring Boot microservices that manage the alert queue and the investigation lifecycle. The APIs must validate incoming data strictly, support filtered queries, and

maintain a tamper-proof audit trail on every case change. The system must start locally with one command.

2. Pre-requisites

2.1 Tools

Tool	Version	Notes
Java	21	Eclipse Temurin. JAVA_HOME pre-configured.
Maven	3.9+	Available on PATH. .m2 cache pre-warmed with all Spring Boot 3.x libraries.
IntelliJ IDEA	Community	Pre-installed with Java 21 SDK configured and Spring Boot run configurations ready.
Docker Desktop (optional)	Latest	docker compose v2 available (docker compose, not docker-compose).
Postman	Latest	Pre-installed. The hackathon collection described in Section 2.4 is already loaded.
Node.js / npm	18 LTS	For the React nice-to-have only.
Git	Latest	Pre-configured with your candidate name. Repository at ~/alertdesk.
curl	Latest	Available in terminal for quick API testing.
H2 Database	Latest	Use H2-In memory DB or any other equivalent RDMS

2.2 Postman Collection

Folder	Requests included
Alert Service	GET /api/alerts (filter combinations), GET /api/alerts/{id}, POST /api/alerts (valid + invalid), PATCH /api/alerts/{id}/status, GET /api/alerts/summary
Case Service	POST /api/cases, GET /api/cases, GET /api/cases/{id}, POST /api/cases/{id}/notes, POST /api/cases/{id}/sar (valid and invalid lifecycle scenarios)
Customer Service	GET /api/customers/{id}, GET /api/customers/{id}/alerts, GET /api/customers/search
Judge Scenarios	Five pre-built requests mapping directly to the five judge test scenarios in Section 10. Run these folder end-to-end to self-check before demo time.

2.3 Seed Data and Reference Files

File	Contents
seed-data/alerts.json	20 synthetic alert records — all three risk bands, all four statuses, all five alert types, amounts £500–£145,000.
seed-data/cases.json	8 synthetic case records — all lifecycle stages, notes, SAR_FILED with rationale, NO_ACTION_TAKEN. Audit log arrays show what yours must look like.
seed-data/customers.json	10 synthetic customer profiles — PEP-flagged, HIGH risk, sanctions match.
seed-data/sample-requests.json	Valid and deliberately invalid JSON request bodies for every write endpoint. Read this before writing your DTO classes.
reference/RestControllerAdvice-example.java	Working @RestControllerAdvice with handlers for Bean Validation failures (400), business rule violations (422), and not-found (404). Copy and adapt.
reference/docker-compose-reference.yml	Annotated docker-compose.yml explaining every directive and common troubleshooting.

 alerts.json
 sample-requests.json
 RestControllerAdvice-example.java
 customers.json
 cases.json

3. System Overview

Two Spring Boot microservices, each with its own H2 in-memory database. Services must not share a database or call each other directly.

Service	Port	Responsibility
Alert Service	8081	Alert queue — create, filter, triage, summarise [Must Have]
Case Management Service	8082	Investigation lifecycle — notes, automatic audit log, SAR decision [Must Have]
Customer Profile Service	8083	Customer context — risk rating, PEP flag, KYC status [Should Have]
React Front-End	3000	Alert Queue Dashboard + Case Detail workbench [Nice to Have]

4. API Specifications

4.1 Alert Service (port 8081)

Endpoint	Method	Description
----------	--------	-------------

/api/alerts	GET	Paginated list with filtering (Section 5). Returns summary DTO shape (Section 7).
/api/alerts/{id}	GET	Full alert detail including flaggedRules.
/api/alerts	POST	Create a new alert. Full Bean Validation (Section 6).
/api/alerts/{id}/status	PATCH	Update status. Valid transitions: NEW → UNDER_REVIEW → ESCALATED → CLOSED. Invalid returns 422.
/api/alerts/summary	GET	Alert counts grouped by groupBy parameter: status (default), riskBand, or alertType.

Alert fields: alertId, customerId, alertType, riskBand (HIGH/MEDIUM/LOW), amount, currency, triggeredAt, status, assignedAnalyst, flaggedRules.

Valid alert types: STRUCTURING, APP_SCAM, SANCTIONS_HIT, UNUSUAL_ACTIVITY, ACCOUNT_TAKEOVER.

4.2 Case Management Service (port 8082)

Every status change and every SAR decision must be written to an audit log entry automatically. Judges will inspect audit log entries directly. Missing entries score zero on Data Integrity.

Endpoint	Method	Description
/api/cases	POST	Open a new case linked to one or more alert IDs. Full Bean Validation.
/api/cases	GET	List cases with optional filtering by status and priority.
/api/cases/{id}	GET	Full case: linked alerts, notes array, auditLog array.
/api/cases/{id}/notes	POST	Add an immutable note. Cannot be edited or deleted once written.
/api/cases/{id}/sar	POST	File SAR decision (FILE or NO_ACTION) with mandatory rationale. Only valid from PENDING_REVIEW.

Case lifecycle — stage skipping returns 422:

OPEN → UNDER_INVESTIGATION → PENDING_REVIEW → SAR_FILED / NO_ACTION_TAKEN / CLOSED

4.3 Customer Profile Service (port 8083) [Should Have]

Endpoint	Method	Description
/api/customers/{id}	GET	Full customer profile: risk rating, PEP flag, KYC status, account numbers.
/api/customers/{id}/alerts	GET	Historical alerts for this customer.
/api/customers/search	GET	Search by name or account number. Query must be at least 3 characters.

5. Filtering

Any filtering implementation that returns correct results is acceptable — Spring Data JPA Specification, @Query annotations, derived query methods, or a custom approach. The outcome matters, not the method.

5.1 Alert Service — GET /api/alerts

All parameters optional. AND logic when multiple are provided.

Parameter	Type	Behaviour
status	String	Filter by: NEW, UNDER_REVIEW, ESCALATED, CLOSED.
riskBand	String	Filter by: HIGH, MEDIUM, LOW.
alertType	String	Filter by one of the valid alert type values.
amountMin	Decimal	Alerts where amount >= this value.
amountMax	Decimal	Alerts where amount <= this value.
page / size	Integer	Zero-based page (default 0), page size (default 20, max 100).
sortBy / sortDir	String	sortBy: triggeredAt (default), amount, status, riskBand. sortDir: desc (default), asc.

5.2 Case Service — GET /api/cases

Two optional parameters, AND logic when both are provided.

Parameter	Type	Behaviour
status	String	Filter by any valid lifecycle stage.
priority	String	Filter by: HIGH, MEDIUM, or LOW.

5.3 Dynamic Summary Endpoint

GET /api/alerts/summary accepts a groupBy parameter. Invalid groupBy returns 400 — not a 500.

groupBy value	Expected response shape
status (default when omitted)	{ "NEW": 8, "UNDER_REVIEW": 4, "ESCALATED": 3, "CLOSED": 5 }
riskBand	{ "HIGH": 8, "MEDIUM": 8, "LOW": 4 }

alertType

```
{ "STRUCTURING": 5, "APP_SCAM": 5, "UNUSUAL_ACTIVITY": 4,
  "SANCTIONS_HIT": 3, "ACCOUNT_TAKEOVER": 3 }
```

6. Input Validation

All write endpoints validate the request body using `@Valid`. If validation fails, the response must list every failing field simultaneously in a single 400 response. Use `RestControllerAdvice-example.java` as your starting point.

6.1 Standard Error Response

```
{
  "timestamp": "2024-03-15T10:32:00Z",
  "status": 400,
  "error": "Validation failed",
  "path": "/api/alerts",
  "violations": [
    { "field": "amount", "message": "must be greater than 0" },
    { "field": "alertType", "message": "must not be blank" }
  ]
}
```

Business rule failures return 422 with the same structure and an empty violations array.

6.2 Validation Rules

Endpoint	Field	Constraint	Error message
POST /api/alerts	customerId	@NotBlank	Customer ID must not be blank
	alertType	@NotBlank + @Pattern	Must be one of the valid alert types
	riskBand	@NotNull + @Pattern	Risk band must be HIGH, MEDIUM, or LOW
	amount	@NotNull + @DecimalMin("0.01")	Amount must be greater than 0
	currency	@NotBlank + @Size(min=3,max=3)	Currency must be a 3-character ISO code
	triggeredAt	@NotNull + @PastOrPresent	Triggered date must not be in the future
	flaggedRules	@NotEmpty	At least one flagged rule must be provided

POST /api/cases	linkedAlertIds	@NotEmpty	At least one alert ID must be linked
	customerId	@NotBlank	Customer ID must not be blank
	priority	@NotNull + @Pattern	Priority must be HIGH, MEDIUM, or LOW
	assignedAnalyst	@NotBlank	Assigned analyst must not be blank
POST .../notes	text	@NotBlank + @Size(min=10,max=2000)	Note text must be between 10 and 2000 characters
	author	@NotBlank	Author must not be blank
POST .../sar	decision	@NotNull + @Pattern	Decision must be FILE or NO_ACTION
	rationale	@NotBlank + @Size(min=20)	Rationale must be at least 20 characters

6.3 Business Rule Violations (422)

- POST /api/cases/{id}/sar when case is not PENDING_REVIEW → 422: SAR can only be filed from PENDING_REVIEW status
- PATCH /api/alerts/{id}/status with invalid transition → 422: Invalid status transition from {current} to {requested}
- GET /api/alerts/summary with unrecognised groupBy → 400: groupBy must be one of: status, riskBand, alertType

7. Response Shapes

Return different amounts of data depending on whether the caller is fetching a list or a single record.

Context	Endpoint	Fields to include
List	GET /api/alerts	alertId, customerId, alertType, riskBand, amount, triggeredAt, status — excludes flaggedRules
Detail	GET /api/alerts/{id}	All fields including flaggedRules
List	GET /api/cases	caseId, status, priority, assignedAnalyst, openedAt, alertCount (integer — not the array)
Detail	GET /api/cases/{id}	All fields including the full notes array and full auditLog array

Pagination metadata in all list responses:

```
{ "content": [...], "page": 0, "size": 20, "totalElements": 20, "totalPages": 1 }
```

8. Containerisation [Optional]

Write a Dockerfile for each service and one docker-compose.yml. Verify locally.

8.1 Dockerfile (two-stage build)

One Dockerfile per service in the service root directory. Update EXPOSE and service name per service:

```
# Stage 1 - build
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn package -DskipTests

# Stage 2 - run
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/*.jar app.jar
EXPOSE 8081
ENTRYPOINT ["java", "-jar", "app.jar"]
```

8.2 docker-compose.yml (local)

Complete the skeleton in the repository root following docker-compose-reference.yml. One service block per microservice, shared bridge network, environment variables for port and application name. Judges will run docker compose up from the repository root — all services must start and seed data must be queryable within 60 seconds.

9. Seed Data

Use the JSON files attached in Sec 2.3(above) to populate H2 on startup. Adapt field names to your entity model.

Dataset	Minimum	Required variety
Alerts	20 records	All three risk bands. All four statuses. All five alert types. Amounts £500-£145,000.

Cases	8 records	All lifecycle stages. Two cases with 2+ notes. One SAR_FILED with rationale. One NO_ACTION_TAKEN.
Customers	10 records	Two PEP-flagged. At least one HIGH risk rating.

10. Deliverables

Must Have

- Alert Service — all five endpoints, seed data, filtering, Bean Validation, groupBy summary
- Case Management Service — all five endpoints, lifecycle enforcement, automatic audit log
- Standard structured error body with violations array on all write endpoints
- Business rule violations returning 422 with descriptive messages
- List endpoints return summary DTO shape, detail endpoints return full shape (Section 7)

Should Have

- Dockerfile per service — two-stage build producing a lean final image
- docker-compose.yml — starts both services with docker compose up from the repository root
- Customer Profile Service — three endpoints
- Swagger / OpenAPI at /swagger-ui.html on each service

Nice to Have

- React Alert Queue Dashboard — KPI cards, filtered table, colour-coded risk bands, inline status update
- React Case Detail workbench — lifecycle stepper, notes feed, SAR Decision panel with PENDING_REVIEW gate
- Custom React hooks, React Context, loading and error states on every API call